CS171 Final Project Report: Predicting Food-at-Home Monthly Prices

Nathan Montero, Arushi Nirmal, Suparn Posina
Department of Computer Science, San Jose State University
{nathan.montero, arushi.nirmal, suparn.posina}@sjsu.edu

Abstract—Food prices are an important economic indicator, and accurate short-term forecasting can benefit both consumers and retailers. We predict the monthly weighted-average unit price of food groups using the USDA Economic Research Service Food-at-Home Monthly Area Prices (F-MAP) dataset. We frame the task as supervised regression and compare a mean baseline with Ridge Regression, a Decision Tree, a Random Forest, a Multi-Layer Perceptron (MLP), and a Long Short-Term Memory (LSTM) network under a strictly chronological train–validation–test split. Models were tuned on a 2017 validation year and then evaluated once on a held-out 2018 test set. All learned models substantially outperform the mean baseline; the Random Forest is the strongest model, reaching a 2018 test RMSE of 0.033, MAE of 0.017, and R^2 of 0.995, with the LSTM a close second. The most interesting insight is that recent price history dominates prediction, while contextual features such as geographic area, calendar month, and the number of reporting stores contribute relatively little at a one-month forecasting horizon.

***Index Terms*—** Machine Learning, regression, food price forecasting, Random Forest, LSTM, USDA F-MAP

I. Introduction

Food prices are a central economic signal, and reliable short-horizon prediction has practical value for consumers planning budgets and for retailers managing pricing and inventory. We study whether the average monthly retail price of a food group can be predicted from its recent price history together with basic context (calendar month, geographic area, and the number of reporting stores). The task is supervised regression. The target is **Unit_value_mean_wtd**, the weighted average unit price of a food group in dollars per 100 grams. Each observation represents one (year, month, food group, geographic area) combination, and the model predicts that combination's price. To address this problem, we compare a mean baseline with five machine learning models: Ridge Regression, Decision Tree, Random Forest, Multi-Layer Perceptron (MLP), and a Long Short-Term Memory (LSTM) network. We evaluate each model using a chronological train-validation-test split and analyze feature importance, prediction errors, and neural network training behavior.

The remainder of this paper is organized as follows: Section II provides background information, Section III describes the dataset and preprocessing pipeline, Sections IV and V present the methodology and experimental setup, Section VI discusses the experimental results and error analysis, and Sections VII and VIII conclude with the discussion and future work.

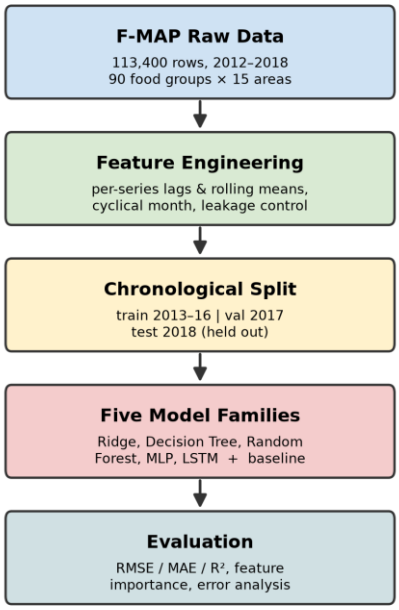

Fig. 1. End-to-end pipeline: data, feature engineering, chronological split, five model families, and evaluation.

II. Related Work / Background

Food price prediction is a supervised regression problem in which historical price observations are used to estimate future prices. Our study uses the USDA ERS Food-at-Home Monthly Area Prices dataset [1], a public collection of retail food prices, and compares standard model families under a single, leakage-controlled evaluation framework rather than proposing a new architecture. As a linear reference we use L2-regularized (Ridge) regression [2]. Among nonlinear models we include the Random Forest [3], an ensemble of decision trees that is robust to feature scaling and captures

interactions automatically, and a Long Short-Term Memory (LSTM) recurrent network [4], which is designed to model temporal dependencies in sequential data. The neural networks are optimized with Adam [5]. Tree and linear models are implemented with scikit-learn [6], and the neural networks use PyTorch [7]. Comparing these families against a mean baseline lets us quantify how much predictive value the engineered temporal features add over a naive predictor.

III. Dataset and Problem Formulation

A. Dataset

We use the F-MAP dataset. It contains 113,400 observations spanning January 2012 through December 2018 (84 months), covering 90 food groups across 15 geographic areas (metro regions plus a National aggregate). The dataset is publicly available from a trusted government source under USDA ERS public-use terms. [1]

Each row contains 15 columns, including identifiers (Year, Month, food-group code/name, region code/name), purchase aggregates (weighted and unweighted purchase dollars and grams), Number_stores, and several price measures, including the target Unit_value_mean_wtd, its standard error, an unweighted mean, and a GEKS price index. The dataset is complete and clean: 0 missing cells and 0 duplicate (Year, Month, food group, area) keys.

Because this is a regression task, class imbalance is not applicable. Instead, we examine the distribution of the continuous target variable and note that it is right-skewed.

Using a 1.5× IQR fence computed per food group, 2,083 of 113,400 observations (1.8%) were flagged as outliers, concentrated in categories such as flavored milk beverages (13.0% flagged) and frozen poultry (10.7% flagged). These were retained rather than removed, since they reflect genuine price variation rather than data-entry errors.

B. Preprocessing

For leakage control we exclude every column that mechanically encodes the price: the purchase-dollar and purchase-gram aggregates (price = dollars ÷ grams), the unweighted mean price, the target's standard error, and the GEKS index. These would trivially reveal the answer. On the feature side, because prices are highly autocorrelated, we add per-series (food group × area) lag and rolling features: price_lag_1, price_lag_3, price_lag_12 and rolling means price_roll_3, price_roll_12. The calendar month is encoded cyclically as month_sin and month_cos so December and January are treated as adjacent. Building a 12-month lag consumes the first year of every series, so 2012 becomes a warm-up year that supplies history but is not itself predicted. This reduces the usable set from 113,400 to 97,200 rows (23 columns after adding the 8 engineered ones). The final model inputs are 12 features: food group and area (categorical), Year, Month, month_sin, month_cos, Number_stores, and the five lag/rolling price features. All preprocessing is fit on the training split only.

C. Problem Statement

The prediction task is formulated as a supervised regression problem. Each training sample represents a single food group and geographic area for a given month, using temporal, geographic, and engineered historical price features to predict the corresponding Unit_value_mean_wtd, the weighted average unit price of a food group. The engineered feature set includes cyclical month encodings (month_sin and month_cos), lagged prices (price_lag_1, price_lag_3, and price_lag_12), and rolling average features (price_roll_3 and price_roll_12). Model performance is evaluated using RMSE, MAE, and R^2.

For the neural network models, we train by minimizing the mean squared error (MSE) between the predicted price $\check{y}$ and the true price y:

$$\mathrm{MSE} = (1/N)\,\Sigma_{i=1}^{N}\,(y_i - \hat{y}_i)^2$$

IV. Methodology

A. Baseline

As a simple reference point, we use a mean predictor that just predicts the training set's average price every time. Every other model has to beat this, or the added complexity is not doing anything useful.

B. Models

Ridge Regression serves as the linear baseline. Categorical features are one-hot encoded and numerical features are standardized before training. The model uses L2 regularization, and the regularization parameter is selected from {0.1, 1, 10, 100} based on validation RMSE. [2] Validation RMSE was essentially flat across $\alpha \in \{0.1, 1, 10, 100\}$ (0.0333 → 0.0333 → 0.0332, best → 0.0334), indicating the linear model was not meaningfully overfitting even at low regularization.

A Decision Tree Regressor is trained using ordinal-encoded categorical variables and the engineered numerical features. Hyperparameter tuning evaluates maximum tree depths of 5, 10, 20, and unlimited depth together with minimum leaf sizes of 1, 5, and 20. The configuration with the lowest validation RMSE is selected.

A Random Forest Regressor is trained using multiple decision trees to improve generalization. Models are evaluated using 50 and 100 trees with maximum depths of 10, 20, and unlimited depth while fixing the minimum leaf size at two samples. The best-performing configuration is selected using the validation set. [3]

The Multi-Layer Perceptron (MLP) is implemented using scikit-learn's MLPRegressor. Candidate architectures include one hidden layer with 64 neurons and two hidden layers with 64 and 32 neurons. The network uses ReLU activation, the Adam optimizer, a learning rate of 0.001, batch size 256,

early stopping, and L2 regularization. Validation performance determines the final architecture. [6]

The LSTM model is implemented in PyTorch. Each sample consists of a 12-month sequence containing standardized price, Number_stores, and cyclical month features. The network uses a single LSTM layer with a hidden size of 48, learned embeddings for food groups (12 dimensions) and geographic areas (4 dimensions), followed by a fully connected layer with 64 hidden units, ReLU activation, 0.15 dropout, and a final linear output layer for regression. [4], [7]

C. Training and Hyperparameters

All models were trained using the chronological split described in Section III, with data from 2013–2016 used for training and 2017 reserved for validation and hyperparameter selection. This approach prevents temporal data leakage while allowing model configurations to be compared consistently before evaluation on the held-out 2018 test set.

Hyperparameters were selected independently for each model family using validation RMSE as the primary criterion. Ridge Regression evaluated multiple values of the L2 regularization parameter (α). Decision Tree and Random Forest models explored different maximum tree depths and model complexities, while the MLP compared multiple hidden-layer architectures together with different L2 regularization strengths. Whichever configuration came out lowest on validation RMSE is what we carried forward to the test set.

The neural network models were trained using the Adam optimizer. The MLP used a learning rate of 0.001, a batch size of 256, and a maximum of 150 epochs with early stopping enabled after ten consecutive epochs without validation improvement. The LSTM also used the Adam optimizer with a learning rate of 0.001, MSE loss, a weight decay of $1x10^{-5}$, training and validation batch sizes of 512 and 1024, respectively, and early stopping with a patience of five epochs. During training, the model automatically restored the weights corresponding to the lowest validation loss. [5]

V. Experimental Setup

The dataset was divided using a **strictly chronological** train/validation/test split to preserve temporal ordering and prevent future information from leaking into the training process. The year 2012 was used as a warm-up period to construct lag features and was therefore excluded from model evaluation. Data from 2013–2016 (64,800 observations) were used for training, 2017 (16,200 observations) was used for validation and hyperparameter selection, and 2018 (16,200 observations) was reserved as the held-out test set for the final evaluation.

We evaluate model performance with root mean squared error (RMSE), mean absolute error (MAE), and the coefficient of determination (R^2): RMSE penalizes big misses more heavily, MAE is easier to read as an average dollar error, and R^2 tells us how much of the variance we are actually explaining.

All experiments were performed using a random seed of 42, which was applied to Python, NumPy, and PyTorch to ensure reproducibility. When CUDA was available, the seed was also applied to all CUDA devices. The MLP and LSTM models were trained using Google Colab, with the LSTM utilizing an NVIDIA Tesla T4 GPU, while the remaining models were trained using scikit-learn.

VI. Results and Analysis

A. Main Results

Model-selection results on the 2017 validation set are reported in Table I, and the final results on the held-out 2018 test set are reported in Table II. All machine-learning models substantially outperform the mean baseline on both splits, which tells us the engineered temporal and contextual features are actually carrying useful signal. The Random Forest achieves the best performance in both cases, obtaining the lowest test RMSE (0.0326) and MAE (0.0174) and the highest test R^2 (0.9950). The LSTM is the closest competitor on the test set, followed by Ridge Regression and the Decision Tree, while the MLP generalizes least well, with its test error noticeably higher than its validation error. Figure 2 plots Random Forest predictions against actual prices on the 2018 test set; the points fall tightly along the diagonal across the full price range.

TABLE I. Model Comparison on the 2017 Validation Set (Best Result in Bold)

Model	RMSE	MAE	R^2
Mean Baseline	0.4592	0.3444	-0.0006
Ridge Regression	0.0332	0.0174	0.9948
Decision Tree	0.0332	0.0183	0.9948
Random Forest	**0.0301**	**0.0161**	**0.9957**
MLP	0.0331	0.0218	0.9948
LSTM	0.0329	0.0179	0.9949

TABLE II. Model Comparison on the 2018 Held-Out Test Set (Best Result in Bold)

Model	RMSE	MAE	R^2
Mean Baseline	0.4631	0.3439	-0.0030
Ridge Regression	0.0362	0.0200	0.9939
Decision Tree	0.0364	0.0196	0.9938
Random Forest	**0.0326**	**0.0174**	**0.9950**
MLP	0.0506	0.0389	0.9880
LSTM	0.0350	0.0196	0.9943

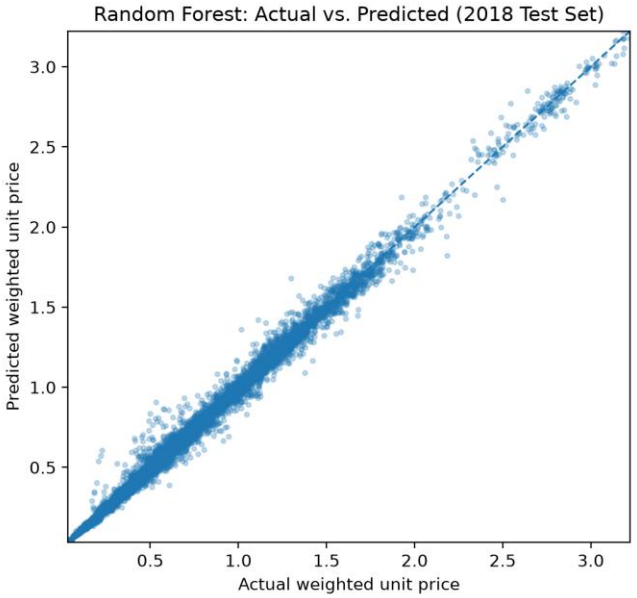

Fig. 2. Random Forest predicted vs. actual price on the 2018 test set.

B. Error Analysis and Ablations

Although all models achieved strong predictive performance, errors were not distributed uniformly across all food categories. Figure 3 presents the Random Forest feature importance, showing that price_lag_1 is by far the most influential predictor. Rolling-average and additional lag features provide smaller contributions, while contextual variables such as geographic area, calendar month, and the number of reporting stores contribute comparatively little. In short, recent price history is the main driver of short-term food price prediction.

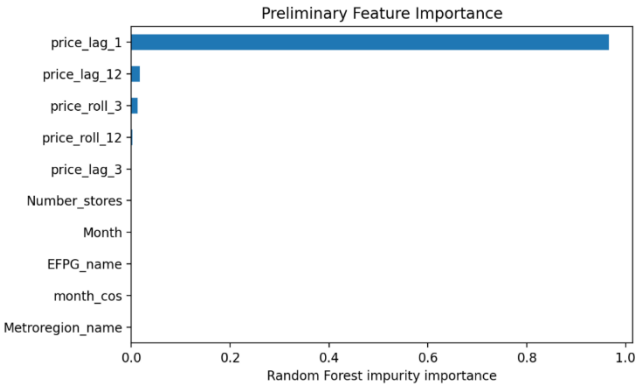

Fig. 3. Random Forest feature importance (price_lag_1 dominates).

Prediction errors are concentrated in more volatile or specialty food categories. As shown in Figure 4, other starchy vegetables (fresh cut), vitamins and meal supplements, dry spices, dark green vegetables (fresh cut), and canned beef, pork, lamb, veal, and game exhibit the highest mean absolute errors. These categories experience greater price volatility, promotional effects, and lower sampling frequency, making future prices less predictable from historical observations alone. In contrast, staple food categories have relatively stable prices and therefore produce substantially lower prediction errors.

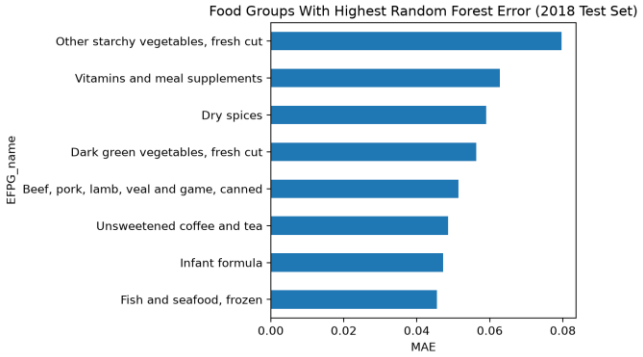

Fig. 4. Food groups with the highest test-set MAE (2018).

VII. Discussion

Every learned model dramatically outperforms the mean baseline, which confirms that the task is learnable and that the engineered features carry real signal. The tree-based and linear models cluster within a narrow band on the validation set, and the Random Forest wins by a small but consistent margin on both the validation and test sets. Feature importance (Figure 3) explains why the models agree so closely: price_lag_1 accounts for the overwhelming majority of the Random Forest's predictive power. Because monthly food prices are highly autocorrelated, next-month price is largely a small adjustment to last month's price, so once recent lags are available the remaining models have little room to differentiate themselves. In bias–variance terms, the low-variance ensemble (Random Forest) and the regularized linear model both land near the achievable error floor for this horizon.

The neural networks behave as expected during training. The LSTM's training and validation MSE both decrease smoothly, stopping early at epoch 24 (of a maximum 30), with no sign of severe overfitting, as shown in the training curve (Figure 5). The MLP, however, is the one model whose test error is markedly worse than its validation error (test RMSE 0.0506 versus 0.0331 on validation), indicating higher variance and mild overfitting to the validation-year distribution; the tree-based models and the LSTM generalize more stably to the unseen 2018 data.

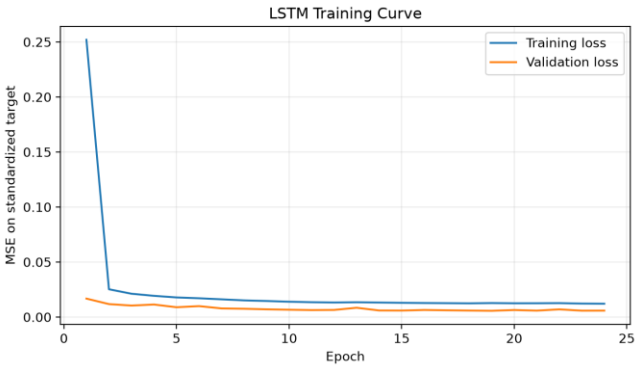

Fig. 5. LSTM training curve: training and validation MSE over 24 epochs (early stopping).

Several limitations temper these results. The near-identical R^2 values across models are partly a consequence of the short one-month horizon and strong autocorrelation, so they should not be read as evidence that model choice is unimportant for harder forecasting problems. Errors concentrate in volatile or specialty categories (Figure 4), where last month's price is a weaker guide. The dataset also spans only 2012–2018 and predates large external shocks such as the COVID-19 period, so generalization to more turbulent regimes is untested. Finally, compute was modest: the LSTM trained in roughly two minutes on a single GPU, so we did not explore larger architectures or extensive neural hyperparameter search. That fits this course's focus on a controlled, leakage-aware comparison rather than squeezing out the last bit of accuracy.

VIII. Conclusion and Future Work

We framed short-horizon food-price prediction on the USDA F-MAP dataset as a supervised regression problem and compared a mean baseline with five model families under a strictly chronological, leakage-controlled split. All learned models beat the baseline by a wide margin, and the Random Forest was the strongest overall, reaching a 2018 test RMSE of 0.033 and R^2 of 0.995. Our central finding is that recent price history—especially the previous month's price—dominates prediction at a one-month horizon, while contextual features add little. Future work includes predicting the month-over-month price change rather than the price level to reduce the dominance of price_lag_1, extending the horizon to several months ahead, adding macroeconomic covariates, evaluating on more recent and more volatile years, and a broader neural hyperparameter search.

Author Contributions

Suparn Posina built the data pipeline and feature-engineering code and set up the reproducible training and evaluation harness. Arushi Nirmal trained and tuned the models, produced the figures and tables, and led the error analysis. Nathan Montero led the writing of the paper and the interpretation of results. We all worked on and published our findings in this report.

References

[1] U.S. Department of Agriculture, Economic Research Service, “Food-at-Home Monthly Area Prices,” 2024. [Online]. Available: https://www.ers.usda.gov/media/5399/food-at-home-monthly-area-prices-2012-to-2018.xlsx

[2] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” Technometrics, vol. 12, no. 1, pp. 55–67, 1970.

[3] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. 3rd Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, 2015.

[6] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 8024–8035.